

NexusPoint — Skills & Projects Catalog

Owner: Aleem Ul Hassan · **System:** Nexis (executive assistant & second brain) · **Generated:** 2026-06-14

This is the full map of what has been built inside Nexis: every custom skill and every project, what problem each one solves, and how to trigger or run it. Skills are invoked in chat by saying the trigger phrase. Projects are codebases you run locally. A short final section lists the installed third-party skills.

Part 1 — Custom Skills

These are the skills built specifically for NexusPoint. Each is invoked by speaking its trigger phrase in a Nexis chat.

A. Sales & Client Acquisition

Sales Playbook

Problem it solves: You need to say the right thing at every stage of cold outreach and closing, without sounding like a templated bot. This is the canonical sales asset that every other outreach tool references. **What it does:** Source-cited opener archetypes (Welsh, Holland, Braun), Voss calibrated questions, Hormozi value equation, Sandler pain funnel, Cole Gordon / Frank Kern closing frameworks, LinkedIn + Instagram cold sequences, a live conversation playbook with objection branches, the full 30-minute Discovery Call (Ops Teardown) script, and 10 cited objection responses. All claims trace to a 77-source NotebookLM research synthesis. The lead offer is AI automation as the premium wedge, never web dev. **How to use:** "draft a DM", "how do I respond to [objection]", "prep me for a sales call", "write a discovery call script", "pitch my AI automation offer", "convert this lead", "what's my opener", or paste a live DM thread and ask "what now".

Cold Outreach

Problem it solves: Getting clients off Upwork/Fiverr requires a repeatable cold email machine, and paid tools are expensive. **What it does:** Runs the full free cold email pipeline end to end. Apify (free tier) scrapes leads, Python finds and verifies emails, Gmail (via gws) sends, and Google Sheets is the CRM. Saves about \$112/month versus paid tools. Manages scraping, email finding, sending today's batch, and reply tracking. **How to use:** "scrape leads", "find emails", "send emails", "send today's batch", "check replies", "outreach status", "run the pipeline".

LinkedIn Outreach

Problem it solves: LinkedIn is a top channel for founders/ops buyers, but personalizing connection notes at scale is slow. **What it does:** Scrapes founder/COO/ops leads via Apify, stores them in a Google Sheets CRM, and generates personalized 300-character connection request notes with OpenAI. **How to use:** "scrape linkedin leads", "generate connection messages", "run linkedin pipeline", "linkedin outreach", "how many linkedin leads".

Instagram Outreach

Problem it solves: Instagram has founder/CEO leads but bans bots, so sending must stay manual while sourcing and personalization are automated. **What it does:** Scrapes founder/CEO/ops profiles via Apify hashtag

search, stores them in a Google Sheets CRM, and generates personalized Touch 1 DMs with OpenAI (GPT-4o-mini). All sending is manual. **How to use:** "scrape instagram leads", "generate instagram DMs", "run instagram pipeline", "instagram outreach", "instagram CRM status".

Marketing Advisor

Problem it solves: You need marketing decisions grounded in current data, not aging frameworks or opinions.

What it does: Research-backed advisor grounded in a NotebookLM synthesis of 234 unique 2026 sources. Covers ICP identification, LinkedIn (organic + outreach), Instagram/Reels, content strategy, email, offer positioning and pricing, paid ads, and marketing automation. Uses Hormozi/Voss only where current evidence supports it. Hands off the actual 1:1 DM/email copy to the sales-playbook skill. **How to use:** "marketing advice", "write a cold email", "LinkedIn strategy", "content plan", "how do I get more clients", "what's a good reply rate", "offer positioning", "pricing", "paid ads", "is X still working", "2026 benchmark".

Discovery Call Prep

Problem it solves: Walking into a prospect call cold loses deals; you need a fast, sharp brief. **What it does:** Given a company name and/or URL, researches the prospect with Firecrawl + web search and outputs a structured prep brief: company snapshot, likely pain points, how to position NexusPoint, sharp questions to ask, and what to watch out for. Works with partial info (just a name or URL). **How to use:** "prep me for a call", "I have a meeting with [company]", "discovery call with", "call prep for", "what should I know about [company]", or paste a URL before a meeting.

AI Use Case Generator

Problem it solves: Prospects want to know what AI can concretely do for *their* business before they'll buy.

What it does: Given a business description, URL, or company + industry, generates 3 tailored AI automation use cases spanning operational efficiency, revenue growth, and customer experience, each with specific ROI metrics and a one-liner hook ready to drop into outreach or a call. **How to use:** "generate AI use cases for [company]", "what AI can I pitch to [prospect]", "give me use cases for [industry]", "prep AI ideas before my call".

Website Audit System

Problem it solves: A free, specific site audit is a powerful cold-outreach hook and also a sellable paid deliverable. **What it does:** Crawls a prospect's site via Firecrawl and runs AI analysis across UX/messaging, SEO, performance (Google PageSpeed Insights), and conversion gaps. Outputs a formatted Google Doc plus a ready-to-send cold email. Two modes: quick (homepage, 3-5 findings + hook email, for outreach) and deep (multi-page crawl, scored per-dimension report, for paid work). **How to use:** "audit this website", "quick audit of [URL]", "deep audit of [URL]", "audit [company]'s site", or paste a URL with "outreach"/"prospect".

Upwork Proposal Generator

Problem it solves: Upwork proposals need to be tailored fast and in your voice to win bids. **What it does:** Generates ready-to-send Upwork proposals from a pasted job post, in Aleem's voice. Triggers automatically whenever job post text is present. **How to use:** Paste a job post, or say "write a proposal", "generate a proposal", "draft a proposal for this job".

Proposal Generator

Problem it solves: Client proposals need a persuasive structure and a polished deliverable, not a blank page.

What it does: Generates client proposals using Hormozi's \$100M Offers framework and creates a formatted Google Doc. **How to use:** "create a proposal for [client]", "draft a proposal".

B. Client Delivery & Operations

Client Onboarding Workflow

Problem it solves: The post-close scramble (folders, docs, checklist, welcome email) eats time and is inconsistent. **What it does:** When a deal closes, spins up the full kit: a Drive folder structure (NexusPoint Clients / [Client] / 5 subfolders), an onboarding Google Doc, a project checklist Google Sheet tailored to the project type, and a Gmail draft of the welcome email (saved as a draft for review, never auto-sent). Can read a prior proposal/discovery Doc to pre-fill. **How to use:** "onboard [client]", "we just signed [client]", "set up onboarding for...", "kick off the [project]", "spin up the folder for [client]", or paste a proposal Doc URL with "set them up".

Client Content Creator

Problem it solves: New clients need their first content suite fast, on-brand, without manual writing. **What it does:** Given a client's brand info/files, produces 5 finished pieces: a 500-700 word SEO blog post, an Instagram caption + hashtags, a LinkedIn post, an 8-slide Instagram carousel with visual concepts, and a 6-8 scene video/reel concept. Packages everything into a PDF and uploads to Google Drive. For CLIENT brands, not Aleem's personal brand. **How to use:** "create content for [client]", "content package for [client]", or paste a client's brand guide and say "turn this into content".

Delegate

Problem it solves: Matching tasks to the right team member and writing the hand-off message is repetitive. **What it does:** Auto-matches a task to the right team member (using the team roster and delegation defaults) and generates a ready-to-send delegation message. **How to use:** "delegate [task]", "assign [task] to [person]", "hand off this task".

Session Closeout

Problem it solves: Decisions and priority shifts get lost at the end of a work session. **What it does:** Wraps up a session: summarizes what was done, extracts decisions worth logging into the decision log, and updates context/current-priorities.md to reflect any shifts. **How to use:** "close out", "wrap up", "end session", "session summary", "what did we do today".

C. Content & Brand

Content Engine

Problem it solves: Consistent personal-brand publishing needs an idea-to-finished-post system, not ad-hoc writing. **What it does:** Full content system for Aleem's personal brand (Instagram, LinkedIn, blog). Pulls ideas from 3 sources (daily-news-brief SQLite, youtube-daily-brief JSON, saved topics sheet), scores them with opportunity scores, researches with OpenAI web search, writes finished content, repurposes across platforms (Blog -> LinkedIn + Instagram), and logs to Google Sheets + Docs. Note: never mentions NexusPoint or university in personal-brand content. **How to use:** "content ideas", "what should I post", "create content", "write a blog", "full content run", "repurpose this", "content calendar".

Daily Brief

Problem it solves: Staying current on AI/tech daily is a manual time sink and a content-idea source. **What it does:** Generates a daily AI/tech intelligence brief. Fetches from NewsAPI, Hacker News, and RSS feeds,

deduplicates, categorizes, analyzes with Claude Haiku per category, and synthesizes across categories with Claude Sonnet. Feeds the content engine. **How to use:** "generate brief", "daily brief", "what's happening in AI today", "AI news today", "run the brief".

Reel Creator

Problem it solves: Static infographic posts have more reach potential as short vertical video, but producing synced motion graphics is tedious. **What it does:** Turns an infographic post (caption + source + image) into a 40-50s vertical motion-graphics reel with an ElevenLabs voiceover synced to brand animation, via the reel-engine codebase. Two phases: (1) authors the voice script + scene content.json (validated for brand voice, length, sync); (2) after you record the voiceover, transcribes with Whisper, aligns scenes, and renders the mp4. **How to use:** "make a reel", "turn this post into a reel", "reel from this infographic", "render the [slug] reel", "the voiceover is ready".

D. Knowledge & Research

Claude Advisor

Problem it solves: Picking the right Claude surface/model and getting the most from Claude Code is non-obvious, and clients ask about it. **What it does:** Research-backed guide to everything Claude: the surfaces (Claude.ai chat, Claude Code, Cowork, Desktop/mobile/API), a "which one is best for this task" decision framework, Claude Code productivity (hooks, MCP, subagents, skills, plugins, Agent SDK), business/creative use cases, the ecosystem, API building, model selection, and plans. Grounded in a NotebookLM synthesis of cited 2026 sources with a live-query fallback. Doubles as the basis for a free lead-magnet guide. **How to use:** "what's the difference between Claude chat and Claude Code", "which Claude should I use for X", "Opus vs Sonnet for this", "best Claude Code plugins", "explain Claude to a client".

Deep Research

Problem it solves: You need reliable research with the right depth/cost trade-off on demand. **What it does:** Context-aware research via OpenAI with deep and quick modes. **How to use:** "research [topic]", or force a mode with "deep research...", "quick search...", "lite research...".

NotebookLM

Problem it solves: Google NotebookLM's full power (including features not in the web UI) is locked behind manual clicking. **What it does:** Full programmatic access to NotebookLM. Create notebooks, add sources (URLs, YouTube, PDFs), chat with content, and generate podcasts, quizzes, reports, flashcards, mind maps, and videos. This is the engine behind the research-backed skills. **How to use:** "create a podcast about X", "list my notebooks", "add this URL to NotebookLM", "generate a quiz from my research", "summarize these documents".

E. University & ML

Assignment Research

Problem it solves: University assignments need credible sources and structure, fast, without mixing into agency work. **What it does:** Researches assignment topics, finds academic sources, synthesizes findings into structured research notes + outlines, and saves them to Google Docs. **How to use:** "research [topic] for my

assignment", "help with my [course] assignment", "find sources for [topic]", "I have an assignment on...".

ML Expert

Problem it solves: AI/ML tasks (coursework or client work) need both advice and runnable, executed code. **What it does:** General-purpose AI/ML expert. Advises on any ML topic and, given an implementation task, generates complete Python code, executes it, and saves both the code file and the run output. Covers EDA, training, evaluation, preprocessing, feature engineering, neural nets, CV, NLP, clustering, regression, classification, plus conceptual questions. **How to use:** "implement [task]", "train a model", "EDA on [dataset]", "build a classifier", "explain overfitting", "which model should I use", or paste an assignment spec.

F. Design

UI Design System

Problem it solves: Visual consistency and clean design-to-dev handoff need tokens and documentation, not guesswork. **What it does:** Toolkit for design tokens, component documentation, responsive design calculations, and developer handoff. Use when creating design systems or maintaining visual consistency. **How to use:** "create a design system", "design tokens for...", "document this component", "responsive calculations", "dev handoff".

Part 2 — Projects

Projects are runnable codebases under [projects/](#). Grouped by purpose.

A. Client Acquisition Systems

Lead Gen ([projects/lead-gen](#))

Problem it solves: Outreach is only as good as the lead list; you need high-quality, scored, enriched prospects across channels. **What it does:** End-to-end lead intelligence pipeline. Discovers prospects via LinkedIn Jobs/Profiles, Product Hunt, and Google Search, scores them with a 5-layer ICP system, enriches with website intel + email + Proxycurl + Perplexity, generates personalized outreach via Claude, and exports to all 3 CRMs. **How to use:** Say "generate leads", "run lead gen", "score leads", "enrich leads", "export leads", "pipeline stats". (Has its own SKILL.md wrapper; runs via [main.py](#).)

Sales Playbook Dashboard ([projects/sales-playbook-dashboard](#))

Problem it solves: Drafting on-brand LinkedIn/Instagram DMs from the playbook needs a fast UI, and the old dashboards drifted into AI-sounding copy. **What it does:** Local + Vercel-ready web app that drafts LinkedIn and Instagram outreach straight from the sales-playbook skill (single source of truth). Platform toggle (LinkedIn/Instagram) x mode toggle (Cold Opener with archetype rotation / Follow-up sequence / Live Reply from a pasted thread). Loads opener archetypes + worked examples and filters against a banned-phrase list so output sounds human. Output shows the message plus the move behind it. **How to run:** [npm install](#), set [ANTHROPIC_API_KEY](#) in [.env.local](#), [npm run dev](#). Model: claude-sonnet-4-6. Sync playbook copies after editing the canonical skill.

Content Engine Dashboard (projects/content-engine-dashboard)

Problem it solves: The content engine needs a visual surface to ideate, generate, and manage posts. **What it does:** Next.js dashboard front-end for the content-engine skill. Platform rules live in both the skill's platform-formats.md and the dashboard route's PLATFORM_SPECS (kept in sync). **How to run:** `npm install, npm run dev`.

Upwork Proposal Dashboard (projects/upwork-proposal-dashboard)

Problem it solves: Generating Upwork proposals is faster with a dedicated UI tied to Gmail/Workspace. **What it does:** Next.js dashboard paired with the upwork-proposal-generator skill. Includes Google OAuth client credentials for Workspace integration. **How to run:** `npm install, npm run dev`.

Upwork Job Scout (projects/upwork-job-scout)

Problem it solves: Manually scanning the Upwork feed for fit is low-leverage; scoring job fit should be automated. **What it does:** Next.js project scaffolded to score job fit against the ICP and draft proposals. Skill layer is the next build step (dashboard scaffolded). **How to run:** `npm run dev`.

Browser Automation (projects/browser-automation)

Problem it solves: Some lead sources and screenshots need real browser scraping. **What it does:** Playwright browser automation scripts and E2E tests, including an Upwork/Framer scraper, a general scraper, and a screenshot utility. **How to run:** Node + Playwright; scripts under `scripts/`, tests via the Playwright config.

Daily News Brief (projects/daily-news-brief)

Problem it solves: The daily-brief skill needs a real pipeline + dashboard behind it. **What it does:** The codebase powering the Daily Brief. NewsAPI (6 queries) + Hacker News (top 150) + RSS (ArXiv, TechCrunch, MIT Tech Review, The Verge, Ars Technica) -> dedup -> categorize -> Haiku per-category -> Sonnet synthesis -> SQLite -> Next.js dashboard. About \$0.06 per run. **How to run:** `npm install`, configure `.env`, `npx drizzle-kit push`, then `npm run generate` (optionally with a date) and run the dashboard.

Reel Engine (projects/reel-engine)

Problem it solves: Producing brand-consistent, voice-synced vertical reels by hand is slow; this is the render system behind reel-creator. **What it does:** Remotion codebase that turns infographic posts into 9:16, 40-50s motion-graphics reels with an ElevenLabs voiceover synced to brand animation (tech-blue gradient identity, Quiche Sans + Urbanist, circuit-N logo). Hybrid native scenes with TikTok-style word-timed captions. Each reel lives in `public/reels/`. **How to run:** Author `content.json`, optionally `npm run studio` to preview, record the voiceover to `voiceover.mp3`, then `node scripts/prepare.mjs` to transcribe + align, then render.

B. Products & Hackathon

BidSense / Bid Engine (projects/bid-engine)

Problem it solves: Bid teams spend 60-80% of their time on manual RFP document drudgery, and one missed mandatory requirement means disqualification. **What it does:** CUST Hackathon 2026, Problem #1 (TEKROWE). Upload an RFP/RFQ/Tender (PDF or DOCX); it extracts every requirement, evaluation criterion, and deadline, checks each requirement against the company's capability library with cited evidence, drafts a

citation-grounded proposal, flags compliance gaps, and scores win probability with an explained GO / CONDITIONAL GO / NO-GO decision in under 5 minutes. Ships with a warmed LLM cache so a cold clone replays the full pipeline offline. Own public repo: Qubit1010/bid-engine. **How to run:** Backend: `pip install -r requirements.txt, uvicorn main:app --port 8000`. Frontend: `npm install, npm run dev`. Or `./setup.ps1`. No API keys needed for the 3-document demo.

C. University & Academic

Website Quality Scorer ([projects/website-quality-scorer](#))

Problem it solves: Manual website evaluation is subjective; this scores and explains site quality with ML (and doubles as portfolio + the audit skill's backbone). **What it does:** ML AIN-373 course project (Aleem + Abdul Hadi Minhas). Full-stack ML app: accepts a URL, crawls ~40 quality features (UX, content, technical, trust), scores 0-100 via XGBoost, explains with SHAP per-feature contributions, and generates ranked recommendations. Stack: Next.js 16 + FastAPI + XGBoost/scikit-learn/SHAP + Firecrawl + PageSpeed. **How to run:** Backend FastAPI + frontend Next.js (see project setup); notebooks for EDA/training/SHAP.

Fraud Detection CCP ([projects/fraud-detection-ccp](#))

Problem it solves: A graded Complex Computing Problem needs an end-to-end, explainable fraud analytics framework. **What it does:** Data Science Lab CCP (BSAI 7th semester). On mobile-money (PaySim) data: EDA -> feature engineering -> imbalance handling (SMOTE/ADASYN/hybrid) -> ML + DL models (LogReg, Decision Tree, Random Forest, XGBoost, ANN, Autoencoder) -> explainability (SHAP/LIME) -> dynamic risk scoring -> unsupervised anomaly discovery (Isolation Forest, LOF, DBSCAN) -> a 4-view BI dashboard, plus a written report (business understanding, scalability, literature review, critical evaluation). **How to run:** `pip install -r requirements.txt`, fetch data, run `notebooks/fraud_detection_ccp.ipynb` top to bottom.

Kalman Filter Assignment ([projects/kalman-filter-assignment](#))

Problem it solves: A coursework assignment on Kalman filtering with reproducible plots and a PDF report. **What it does:** Python implementation of a Kalman filter with generated plots (position estimate, covariance evolution, Kalman gain, effect of Q and R, innovation) and a markdown-to-PDF report pipeline. **How to run:** `python kalman_filter.py`, then `python md_to_pdf.py` to build report.pdf.

Robotics Maze Solver ([projects/robotics-maze-solver](#))

Problem it solves: A Webots lab project demonstrating real autonomous navigation without a map. **What it does:** A TurtleBot3 Burger with 360-degree LiDAR autonomously crosses a two-room obstacle environment to a goal beacon using a gap-seeking (VFH-style) controller: clearance-aware avoidance, boxed-in recovery, and odometry path drawing. No map, no teleop. **How to run:** Open `worlds/lidar_nav.wbt` in Webots; controller in `controllers/lidar_nav/`.

Swarm Flocking ([projects/swarm-flocking](#))

Problem it solves: A Webots lab project demonstrating emergent multi-agent behavior. **What it does:** Five e-puck robots run the same controller and exhibit collective flocking via Reynolds Boids (separation, cohesion, alignment) read from the Webots scene tree, with IR proximity wall avoidance. Order emerges from local peer rules; no robot has a goal. **How to run:** Open the Webots world; identical controllers run in parallel on each robot.

Data Science Case Studies Presentation

([projects/data-science-case-studies-presentation](#))

Problem it solves: A presentation deliverable needing real, cited case studies of data science impact. **What it does:** A presentation pack ("Data Science Applications in the Real World") with detailed response + speaker notes, researched via NotebookLM (10 sources), with every number traced to a source across finance, healthcare, and operations. **How to use:** [presentation-pack.md](#) + the speaker-notes PDF.

Responsible Data Scientist Presentation

([projects/responsible-data-scientist-presentation](#))

Problem it solves: A presentation on responsible/ethical data science with a structured script and report. **What it does:** A Gamma outline, opening script, and a markdown + PDF report for a presentation on responsible data science. **How to use:** [gamma-outline.md](#), [opening-script.md](#), [report.pdf](#).

D. Experimental

LightRAG ([projects/lightrag](#))

Problem it solves: Placeholder for a graph/light RAG experiment. **What it does:** Currently empty — reserved/experimental directory, no implementation yet.

Part 3 — Installed Third-Party Skills

These ship from plugins/marketplaces (not custom-built for NexusPoint) but are available in this workspace:

- **agent-development** — guidance on building Claude Code agents (frontmatter, system prompts, triggering).
 - **brainstorming** — structured intent/requirements exploration before any creative build.
 - **canvas-design** — create visual art as .png/.pdf (posters, designs).
 - **code-reviewer** — code review across TS/JS/Python/Swift/Kotlin/Go with security scanning.
 - **frontend-design** — distinctive, production-grade frontend UI generation.
 - **remotion-best-practices** — best practices for Remotion video creation (used by the reel work).
 - **senior-architect / senior-backend / senior-frontend** — architecture, backend, and frontend engineering skills.
 - **skill-creator** — create, edit, optimize, and eval skills.
 - **ui-ux-pro-max** — broad UI/UX design intelligence (styles, palettes, font pairings, stacks).
 - **Stitch suite** (design-md, enhance-prompt, react-components, remotion, stitch-design, stitch-loop, shadcn-ui) — Stitch design-to-code workflow.
-

End of catalog.